gstack 项目详解（中文）

> 说明：本文基于当前工作区快照整理，目标是用中文解释这个仓库到底是什么、怎么工作、该从哪里读起。 如果你更关心每个 skill 的用途，请继续看 docs/skills.zh-CN.md。

一句话认识这个项目

gstack 不是单个工具，也不是一份 prompt 集合。它更像一个“给 AI agent 用的软件工程工厂”：

> 一部分是**工作流技能层**：用很多 SKILL.md 把一个代理拆成 CEO、工程经理、设计师、评审、QA、发布工程师、调试器等不同角色；
>
> 另一部分是**浏览器运行时层**：提供一个持久化、低延迟、可脚本化的 Chromium，让代理真的能去打开页面、点击按钮、看截图、读控制台和抓网络请求。

项目真正想解决的问题是：把“一个会写代码的代理”变成“一个有流程、有关卡、有角色分工的虚拟工程团队”。

这个仓库的核心价值

1. 它把“提示词”升级成“流程”

很多 AI coding 项目的问题在于，代理一上来就写代码，缺少需求澄清、方案评审、设计审查、测试、上线和复盘这些工程环节。gstack 的设计不是“给一个更长的 prompt”，而是把完整 sprint 拆成一条明确链路：

Think -> Plan -> Build -> Review -> Test -> Ship -> Reflect

你在 README 里看到的 /office-hours、/plan-ceo-review、/plan-eng-review、/review、/qa、/ship、/retro，其实就是把这条链路编码成一组可调用的 skill。

2. 它最硬核的技术资产不是 Markdown，而是浏览器 runtime

从实现角度看，gstack 最关键的工程部分不是那些 prompt，而是 browse/。原因很简单：

> 如果每次工具调用都重新启动浏览器，AI 做 QA 会非常慢；
>
> 如果浏览器每次调用后就消失，cookie、tab、登录态和 localStorage 都留不住；
>
> 如果代理看不到真实页面、点不了真实按钮，它很多“会修 bug”的能力其实是假的。

所以 gstack 采用的是**浏览器守护进程**模型：

> 首次调用时启动 Chromium；
>
> 之后所有命令都走本地 HTTP 通道；
>
> 浏览器状态保留，后续命令一般只要 100 到 200ms。

这也是为什么仓库里 browse/ 目录的工程含量很高，而其他大多数 skill 本质上是 Markdown 工作流。

代码库怎么组织

从顶层结构看，这个仓库可以拆成 6 层：

层	目录/文件	作用
技能源码层	*/SKILL.md.tmpl	真正应该编辑的 skill 模板
技能生成层	*/SKILL.md、SKILL.md	由模板和脚本生成的英文技能文件
宿主分发层	.agents/skills/*	面向 Codex/agents 宿主的输出目录
浏览器运行时	browse/	持久化无头浏览器 CLI、服务端、命令分发和测试
工具脚本层	scripts/	生成技能文档、检查健康状态、统计和评估
测试与评估层	test/、browse/test/	单测、静态校验、E2E、LLM 评估

你现在看到的 skill 文件并不都处在同一层

这是这个项目最容易让人读混的地方。

源码层

例如：

> **office-hours/SKILL.md.tmpl**
> **review/SKILL.md.tmpl**
> **browse/SKILL.md.tmpl**

这些模板才是要维护的源文件。

生成层

例如：

> **office-hours/SKILL.md**
> **review/SKILL.md**
> **SKILL.md**

这些是从模板生成出来的英文 skill 文档，不应该直接手改。

宿主分发层

例如：

> **.agents/skills/gstack-office-hours/SKILL.md**
> **.agents/skills/gstack-review/SKILL.md**

这一层是为 Codex/agents 宿主准备的输出。它和源码层不是简单复制关系，而是经过了宿主路径、提示前缀等适配。

生成链路是这个仓库的主骨架之一

根目录 scripts/gen-skill-docs.ts 是整个技能系统的生成器。它做的事情可以理解为:

SKILL.md.tmpl -> 读取占位符 -> 从源码提取信息 -> 生成英文 SKILL.md -> 按宿主生成 .agents/skills 输出

这个脚本做了几件重要的事:

1. 识别当前目标宿主是 Claude 还是 Codex。
1. 为不同宿主注入不同路径，例如 ~/.claude/skills/... 和 .agents/skills/...。
1. 生成通用 preamble，包括:
   升级检查
   会话追踪
   AskUserQuestion 统一格式
   Completeness Principle
   contributor mode / telemetry 逻辑
1. 对 /browse 相关技能，从 browse/src/commands.ts 和 browse/src/snapshot.ts 里动态提取命令和 flag 文档。

这个设计很关键，因为它避免了“代码支持某个命令，但技能文档忘记更新”的文档漂移问题。

browse/ 子系统为什么这么重要

如果你只把 gstack 当成一组 skill，很容易忽略 browse/ 其实是一个独立的工程系统。根据 ARCHITECTURE.md，它大致是这样工作的:

1. CLI 二进制读取状态文件，知道当前浏览器守护进程在哪个端口。
1. CLI 通过 localhost HTTP 把命令发给服务端。
1. 服务端再通过 Playwright/Chromium 执行真实浏览器操作。
1. 浏览器本身常驻，保留 cookies、tabs、session、localStorage。

关键设计点

守护进程模型

第一次启动浏览器大约要几秒;
后续命令就只是本地请求，不必重启浏览器;
这对 QA、回归测试和长流程操作特别重要。

@e1 / @c1 引用系统

gstack 不要求代理去手写 CSS selector 或 XPath，而是让它先跑 snapshot，拿到一棵带编号的交互树，然后再用 @e3 这样的引用去点击或填写。

这个设计的好处是:

对 LLM 更友好;

不依赖 DOM 注入；

避免在 React/Vue/Shadow DOM 场景里因为选择器脆弱而失效。

本地安全模型

浏览器服务只绑定到 localhost，并要求 bearer token。cookie 解密发生在进程内存里，不把明文写回磁盘。这说明作者对“AI 能访问浏览器”这件事是有安全边界意识的。

skill 不是平铺工具，而是被组织成一条研发流程

如果你只看目录，会误以为这是 20 多个散乱技能。实际上它们有明显的角色分工和调用顺序：

1. 想法与范围层

/office-hours

/plan-ceo-review

/plan-eng-review

/plan-design-review

/design-consultation

这一层的目标不是写代码，而是搞清楚做什么、为什么做、怎么设计、怎么实现、怎么验证。

2. 实施与质量层

/review

/investigate

/design-review

/qa

/qa-only

这一层在处理“已经动手了之后，如何保证质量、排查问题、验证体验”。

3. 发布与回顾层

/ship

/document-release

/retro

这一层对应的是交付闭环：发布、同步文档、回顾过程。

4. 浏览器与安全辅助层

/browse

/setup-browser-cookies

/careful

/freeze
/guard
/unfreeze
/gstack-upgrade
/codex

这一层要么提供底层能力，要么提供附加安全性与第二视角。

测试体系说明了作者把“skill 设计”当成真正工程

CLAUDE.md 和 ARCHITECTURE.md 里有一个很重要的信号：这个仓库不是凭感觉改 prompt，而是给 skill 设计了明确测试层级。

Tier 1：静态校验

解析 SKILL.md
校验命令是否合法
校验生成结果是否新鲜

这部分便宜、快，适合每次提交都跑。

Tier 2：E2E

真的拉起 Claude/Codex 会话
真的跑技能流程
看会不会出错、会不会跑偏

这部分更接近真实使用成本，但更贵更慢。

Tier 3：LLM Judge

让模型评估技能文档的清晰度、完整性、可执行性

这部分很像“prompt 工程的评测层”。

这套设计说明作者把 skill 文档当成一种**需要测试、需要回归、需要评估的工程资产**，而不是随手写出来的说明文本。

为什么这个仓库会同时出现 README、AGENTS、CLAUDE、ARCHITECTURE

因为它面向的不只是一个读者群体：

README.md：面向使用者和潜在用户，强调价值主张和上手方式。
AGENTS.md：面向代理宿主，告诉代理在这个仓库里如何协作。

CLAUDE.md：面向贡献者或 Claude 宿主，说明开发约定、命令和结构。

ARCHITECTURE.md：面向想真正理解实现原因的人。

换句话说，这个仓库既是产品，也是开发工具，也是 prompt 工程系统，所以文档被拆成多种角色视角。

你阅读这个项目时最应该抓住的 8 个点

1. **这是流程产品，不只是技能列表。**

真正的产品是“一套 AI 工程工作流”，skill 只是这个流程的界面。

1. **浏览器 runtime 是技术核心。**

没有持久化浏览器，很多 QA、dogfooding、回归验证都不成立。

1. **模板才是源码。**

SKILL.md.tmpl 才是要维护的源文件，SKILL.md 是生成物。

1. **生成器是仓库基础设施。**

scripts/gen-skill-docs.ts 不是辅助脚本，而是核心生产线。

1. **多宿主适配是这个项目的重要设计点。**

Claude、Codex/agents 的路径、分发位置和运行环境不同，所以项目做了宿主层输出。

1. **安全与边界是内建能力。**

/careful、/freeze、/guard 说明作者在主动抑制代理“越权”或“误操作”的问题。

1. **测试体系比很多 prompt 仓库严肃。**

这里不仅测试 TypeScript，也测试 skill 文档本身的可执行性和效果。

1. **它把 prompt 视为工程资产。**

有模板、有生成、有测试、有评估、有宿主适配，这本质上是在工程化 prompt/workflow。

当前仓库快照里一个值得注意的细节

在当前工作区里：

根目录源码层可以看到 codex/ skill；

但 .agents/skills/ 里暂时没有对应的 gstack-codex 输出目录。

这说明你在读这个仓库时，不能假设“源码层目录”和“宿主分发层目录”永远完全一一对应。某些 skill 可能只存在于特定宿主，或者当前快照还没同步到另一层。

对你当前诉求最有用的阅读路径

如果你是第一次看这个仓库，建议按这个顺序读：

1. README.md

先理解作者想把它卖给谁、解决什么问题。

1. docs/project-overview.zh-CN.md

搭建整体理解框架。

1. docs/skills.zh-CN.md

再去看每个 skill 在这条流程里扮演什么角色。

1. 某个具体目录下的 SKILL.md.tmpl + 同目录 SKILL.zh-CN.md

一边看英文源码，一边用中文阅读版对照。

1. scripts/gen-skill-docs.ts

搞懂模板怎么变成最终 skill 文档。

1. ARCHITECTURE.md 和 browse/

最后理解为什么浏览器层要这样设计。

本次补充的中文文件是怎么定位的

为了不改动英文原文件，我新增的是“中文阅读辅助层”：

> 每个 skill 目录新增 SKILL.zh-CN.md
> 根目录新增 SKILL.zh-CN.md
> 总览新增：
> docs/skills.zh-CN.md
> docs/translation-index.zh-CN.md
> memory.md

其中：

> 英文文件仍然负责被宿主读取和执行；
> 中文文件只负责帮助你读懂项目；
> memory.md 记录了今后英文 skill 变化后要同步更新中文版本的约定。

总结

如果用一句更工程化的话概括 gstack：

它是一个把 AI 代理工作流“产品化、模板化、生成化、测试化、宿主适配化”的仓库。

表面上你看到的是很多 SKILL.md；本质上它在做三件事：

1. 用角色分工替代单一代理乱冲。
1. 用持久化浏览器给代理真实世界的可观测性和可操作性。
1. 用模板、生成器和测试体系把 prompt/workflow 当成真正的软件资产来维护。